

FRED TALKS

2nd May 2026

CONTENTS

"Collaboration" is bullshit.

WESTENBERG. · 1380 WORDS

Getting Real About Distributed System Reliability

BOREDANDROID · 2792 WORDS

It's-a Me, Agentic AI

HAN, NOT SOLO · 2362 WORDS

Bottle

XKCD.COM · 19 WORDS

"Collaboration" is bullshit.

WESTENBERG. · 29 MAR 2026 · [SOURCE](#)



This newsletter is free to read, and it'll stay that way. But if you want more - extra posts each month, access to the community, and a direct line to ask me things - paid subscriptions are \$2.50/month. A lot of people have told me it's worth it.

Upgrade

In 1944, the Wehrmacht launched into Hitler's last ditch effort to save the Third Reich. The Battle of the Bulge was a doomed campaign and a doomed gamble from a doomed regime, but its brutality was a true second test of the US Army on the Western Front. During the battle, Army historian S.L.A Marshall began interviewing infantry companies who'd been baptised in combat. Published 3 years later in his 1947 book, *Men Against Fire*, Marshall's research showed that just 15-20% of riflemen in active combat positions ever fired their weapons - most kept their heads down. They moved when they were ordered and they held their positions, and they mimicked the outward appearance of a soldier in battle - but shoot, they did not. By any standard organisational metric, the men were present and accounted for, but 4 out of 5 never pulled the trigger.

You can debate the extent of Marshall's numbers, and you can debate his methodology, but his ratio shows up, again and again. IBM stumbled onto it in the '60s when they discovered that 80% of computer usage came from 20% of the system's features. The pattern recurs because it describes something real about how effort is distributed inside groups, where a fraction of the people do most of the work, and the rest provide what you might ~charitably call "structural support."

Anyone who has worked in any large organisation knows exactly what I'm talking about.

The modern tech industry looked at the problem of human coordination and participation and decided the solution was “collaboration.” If only 20% of us are operating with a “killer instinct” we need to be better at managing the shared instincts of the other 80%. And so collaboration became our shared obsession. We pursue “teamwork” as a holy grail.

The teamwork revolution, if you can call it that, gave us Notion for our documents, ClickUp for our tasks, Slack for our conversations, Jira for our tickets, Monday for our boards, Teams for the calls that should been emails, emails for the things that we couldn’t squeeze in anywhere else, and now agents attempting to re-invent the whole stack. The average knowledge worker maintains accounts across system after system, switching between applications hundreds of times per day. And they produce, in aggregate, a staggering amount of coordinated and collaborative activity that never actually becomes anything resembling ~output.

When you strip away the product marketing and the dev relations and the blog posts and the funding rounds and the fuckery-upon-fuckery of it all, we’re left with a simulation of collective engagement - but very little else. Transparency got confused with progress, visibility got confused with accountability, and being included in the thread became the same thing, socially and organizationally, as owning the outcome.

Once that confusion set in at the cultural level it became nearly impossible to dislodge. The feeling of collaboration is pleasant in a way that personal accountability can never be. Owning something means you, specifically and visibly you, can fail at it, specifically and visibly, in ways that attach to your name.

Collaborating means the failure belongs to the process.

So everyone chose collaboration, and we called it culture.

Marshall's riflemen were ordinary people responding to the diffusion of responsibility that happens inside any group. Maximilien Ringelmann measured the same phenomenon with ropes in 1913, long before there were Slack workspaces to offer an emoji-react to it. Individual effort drops predictably as group size increases. The presence of others dissolves the sense of personal responsibility in a way that feels, to everyone experiencing it, entirely reasonable. You're part of a team, you're contributing, you're also (measurably) pulling less hard than you would if the rope were yours alone. Every single person on the rope is doing this simultaneously, which is why the total force never adds up the way the headcount says it should.

Frederick Brooks identified the same dynamic in software development in 1975, watching IBM's System/360 project illustrate his emerging thesis that adding people to a late project makes it later. Communication overhead grows faster than headcount, coordination costs

compound, and every new person contributes their capacity along with their relationships to everyone else. Those relationships require maintenance and produce misalignment and generate the need for more meetings to address the misalignment those meetings created.

Brooks might as well have described your company's Q3 roadmap planning cycle and your startup's sprint retrospective, all of which have gotten longer every year and produced, relative to their investment, less.

The collaboration industry has spent a fortune obscuring a dirty truth: most complex, high-quality work is done by individuals or very small groups operating with clear authority and sharp accountability, then rationalized into the language of teamwork afterward. Dostoevsky wrote *The Brothers Karamazov* alone. The Apollo Guidance Computer came from a team at MIT small enough to have real ownership, hierarchical enough that Margaret Hamilton's name could go on the error-detection routines she personally designed.

Communication matters, and shared context matters. But there's a huge difference between communication and collaboration as infrastructure to support individual, high-agency ownership, and communication and collaboration as the primary activity of an organisation. Which, if we're honest, is what most collaboration-first cultures have actually built. They've constructed extraordinarily sophisticated machinery for the social management of work, without actually doing the work they're socialising about.

If and when it exists, ownership looks like an individual who deeply gives a shit, making a call without waiting for group-consensus. That individual will be right sometimes, and they'll be wrong other times, and they'll own it. They won't sit around waiting to find out who has the authority to move a card from one column to another and post about it in the #celebrations channel.

But being that person sucks when "collaboration" is the reigning value, because every unilateral decision gets read as a cultural violation and a signal that you aren't a team player. Collaboration-as-ideology has made ownership and responsibility feel antisocial, which is a hell of a thing, given that ownership is the only mechanism that gets anything across the finish line.

You can see this excess everywhere. Standups where people announce their busy work and as long as everyone's "on the same page" nobody changes course. Documents that are written to perform thinking so somebody else can perform thinking, with no decision in sight. Retros, and kickoffs, and WIP meetings that spawn their own retros, kickoffs and WIP meetings like cells dividing and re-dividing, with zero connection to the work that it's nominally organising around.

Every project now seems to carry more coordination overhead than execution time, and when it fails the postmortem just recommends more collaboration...

At some point (and I think that point was fucking yesterday) we have to ask ourselves - what are we actually producing and who is actually responsible for producing it?

Because at some level, the answer for “who is responsible for X” has to be one single person, no matter how much the collaborative apparatus layered over modern work has been engineered to make that person invisible and dissolve accountability.

We need to find some path back to trusting that individuals will do their jobs, without every responsibility being visible to an entire organisation, without follow-ups being scheduled by a cadre of overpaid managers with their overfed metrics.

Maybe - just maybe - we could make our lives a little easier. Maybe we could let human beings keep their own lists of tasks, and we could let them sink or swim by how they manage those tasks, and we could assign blame to them and to them alone when they fuck up. Maybe we could do it without needing to have team-level views of every Kanban, calendar and task list. And maybe - if we let go of the warm, expensive fiction of collective endeavour - we could make it a little easier to see who among us are pulling the trigger and who are just keeping their heads down.

Getting Real About Distributed System Reliability

BOREDANDROID · 01 MAR 2026 · [SOURCE](#)

There is a lot of hype around distributed data systems, some of it justified. It's true that the internet has centralized a lot of computation onto services like Google, Facebook, Twitter, LinkedIn (my own employer), and other large web sites. It's true that this centralization puts a lot of pressure on system scalability for these companies. It's true that incremental and horizontal scalability is a *deep* feature that requires redesign from the ground up and can't be added incrementally to existing products. It's true that, if properly designed, these systems can be run with no *planned* downtime or maintenance intervals in a way that traditional storage systems make harder. It's also true that software that is explicitly designed to deal with machine failures is a very different thing from traditional infrastructure. All of these properties are critical to large

web companies, and are what drove the adoption of horizontally scalable systems like [Hadoop](#), [Cassandra](#), [Voldemort](#), etc. I was the original author of Voldemort and have worked on distributed infrastructure for the last four years or so. So in-so-far as there is a “big data” debate, I am firmly in the “pro-” camp. But one thing you often hear is that this kind of software is more reliable than the traditional alternatives it replaces, and this just isn’t true. It is time people talked honestly about this.

You hear this assumption of reliability everywhere. Now that scalable data infrastructure has a marketing presence, it has really gotten bad. Hadoop or Cassandra or what-have-you can tolerate machine failures then they must be unbreakable right? Wrong.

Where does this come from? Distributed systems need to partition data or state up over lots of machines to scale. Adding machines increases the probability that some machine will fail, and to address this these systems typically have some kind of replicas or other redundancy to tolerate failures. The core argument that gets used for these systems is that if a single machine has probability P of failure, and if the software can replicate data N times to survive $N-1$ failures, and if the machines fail *independently*, then the probability of losing a particular piece of data must be P^N . So for any desired reliability R and any single-node failure probability P you can pick some replication N so that $P^N < R$. This argument is the core motivation behind most variations on replication and fault tolerance in distributed systems. It is true that without this property the system would be hopelessly unreliable as it grew. But this leads people to believe that distributed software is somehow innately reliable, which unfortunately is utter hogwash.

Where is the flaw in the reasoning? Is it the [dreaded Hadoop single points of failure](#)? No, it is far more fundamental than that: the problem is the assumption that failures are independent. Surely no belief could possibly be more counter to our own experience or just common sense than believing that there is no correlation between failures of machines in a cluster. You take a bunch of pieces of identical hardware, run them on the same network gear and power systems, have the same people run and manage and configure them, and run the same (buggy) software on all of them. It would be incredibly unlikely that the failures on these machines would be independent of one another in the probabilistic sense that motivates a lot of distributed infrastructure. If you see a bug on one machine, the same bug is on all the machines. When you push bad config, it is usually game over no matter how many machines you push it to.

P^N is an upper bound on reliability but one that you could never, never approach in practice. For example Google has a [fantastic paper](#) that gives empirical numbers on system failures in Bigtable and GFS and reports empirical data on groups of failures that show rates several orders of magnitude higher than the independence assumption would predict. This is what one of the best system and operations teams in the world can get: your numbers may be far worse.

The actual reliability of your system depends largely on how bug free it is, how good you are at monitoring it, and how well you have protected against the myriad issues and problems it has. This isn't any different from traditional systems, except that the new software is far less mature. I don't mean this disparagingly, I work in this area, it is just a fact. Maturity comes with time and usage and effort. This software hasn't been around for as long as MySQL or Oracle, and worse, the expertise to run it reliably is much less common. MySQL and Oracle administrators are plentiful, but folks experience with, say, serious production Zookeeper operations knowledge are much more rare.

Kernel filesystem engineers say it takes about a decade for a new filesystem to go from concept to maturity. I am not sure these systems will be mature much faster—they are not easier systems to build and the fundamental design space is much less well explored. This doesn't mean they won't be *useful* sooner, especially in domains where they solve a pressing need and are approached with an appropriate amount of caution, but they are not yet by any means a mature technology.

Part of the difficulty is that distributed system software is actually quite complicated in comparison to single-server code. Code that deals with failure cases and is “cluster aware” is extremely tricky to get right. The root of the problem is that dealing with failures effectively explodes the possible state space that needs testing and validation. For example it doesn't even make sense to expect a single-node database to be fast if its disk system suddenly gets really slow (how could it), but a distributed system does need to carry on in the presence of single degraded machine because it has some many machines, one is sure to be degraded. These kind of “semi-failures” are common and very hard to deal with. Correctly testing these kinds of issues in a realistic setting is brutally hard and the newer generation of software doesn't have anything like the QA processes its more mature predecessors had. (If you get a chance get someone who has worked at Oracle to describe to you what kind of testing they do to a line of code that goes into their database before it gets shipped to customers). As a result there are a lot of bugs. And of course these bugs are on all the machines, so they absolutely happen together.

Likewise distributed systems typically require more configuration and more complex configuration because they need to be cluster aware, deal with timeouts, etc. This configuration is, of course, shared; and this creates yet another opportunity to bring everything to its knees.

And finally these systems usually want lots of machines. And no matter how good you are, some part of operational difficulty always scales with the number of machines.

Let's discuss some real issues. We had a bug in [Kafka](#) recently that lead to the server incorrectly interpreting a corrupt request as a corrupt log, and shutting itself down to avoid appending to a corrupt log. Single machine log corruption is the kind of thing that should happen due to a disk

error, and bringing down the corrupt node is the right behavior—it shouldn't happen on all the machines at the same time unless all the disks fail at once. But since this was due to corrupt *requests*, and since we had one client that sent corrupt requests, it was able to sequentially bring down all the servers. Oops. Another example is [this Linux bug](#) which causes the system to crash after ~200 days of uptime. Since machines are commonly restarted sequentially this lead to a situation where a large percentage of machines went hard down one after another. Likewise any memory management problems—either leaks or GC problems—tend to happen everywhere at once or not at all. Some companies do public post-mortems for major failures and these are a real wealth of failures in systems that aren't supposed to fail. [This paper](#) has an excellent summary of HDFS availability at Yahoo—they note how few of the problems are of the kind that high availability for the namenode would solve. This list could go on, but you get the idea.

I have come around to the view that the real core difficulty of these systems is operations, not architecture or design. Both are important but good operations can often work around the limitations of bad (or incomplete) software, but good software cannot run reliably with bad operations. This is quite different from the view of unbreakable, self-healing, self-operating systems that I see being pitched by the more enthusiastic NoSQL hypesters. Worse yet, you can't easily buy good operations in the same way you can buy good software—you might be able to hire good people (if you can find them) but this is more than just people; it is practices, monitoring systems, configuration management, etc.

These difficulties are one of the core barriers to adoption for distributed data infrastructure. LinkedIn and other companies that have a deep interest in doing creative things with data have taken on the burden of building this kind of expertise in-house—we employ committers on Hadoop and other open source projects on both our engineering and operations team, and have done a lot of [from-scratch development](#) in this space where there was gaps. This makes it feasible to take full advantage of an admittedly valuable but immature set of technologies, and let's us build products we couldn't otherwise—but this kind of investment only makes sense at a certain size and scale. It may be too high a cost for small startups or companies outside the web space trying to bootstrap this kind of knowledge inside a more traditional IT organization.

This is why people should be excited about things like Amazon's [DynamoDB](#). When DynamoDB was released, the company DataStax that supports and leads development on Cassandra released [a feature comparison checklist](#). The checklist was unfair in many ways (as these kinds of vendor comparisons usually are), but the biggest thing missing in the comparison is that *you* don't run DynamoDB, Amazon does. That is a huge, huge difference. Amazon is good at this stuff, and has shown that they can (usually) support massively multi-tenant operations with reasonable SLAs, *in practice*.

I really think there is really only one thing to talk about with respect to reliability: continuous hours of successful production operations. That's it. In some ways the most obvious thing, but not typically what you hear when people talk about these systems. I will believe the system can tolerate (some) failures when I see it tolerate those failures; I believe it can run for a year without downtime when I see it run for a year without downtime. I call this empirical reliability (as opposed to theoretical reliability). And getting good empirical reliability is really, really hard. These systems end up being large hunks of monitoring, tests, and operational procedures with a teeny little distributed system strapped to the back.

You see this showing up in discussions of the CAP theorem all the time. The CAP theorem is a useful thing, but it applies more to system design than system implementations. A design is simple enough that you can maybe prove it provides consistency or tolerates partition failures under some assumptions. This is a useful lens to look at system designs. You can't hope to do this kind of proof with an actual system implementation, the thing you run. The difficulty of building these things means it is really unthinkable that these systems are, in actual reality, either consistent, available, *or* partition tolerant—they certainly all have numerous bugs that will break each of these guarantees. I really like this paper that takes the approach of actually trying to calculate the observed consistency of eventually consistent systems—they seem to do it via simulation rather than measurement, which is unfortunate, but the idea is great.

It isn't that system design is meaningless, it is worth discussing the system design as it does act as a kind of limiting factor on certain aspects of reliability and performance as the implementation matures and improves, but don't take it too seriously as guaranteeing *anything*.

So why isn't this kind of empirical measurement more talked about? I don't know. My pet theory is it has to do with the somewhat rigid and deductive mindset of classical computer science. This is inherited from pure math, and conflicts with the attitude in scientific disciplines. It leads to a preference for starting with axioms, and then proving various properties that follow from these axioms. This world view doesn't embrace the kind of empirical measurements you would expect to justify claims about reality (for another example see this great blog post on programming language productivity claims in programming language research). But that is getting off topic. Suffice it to say, when making predictions about how a system will work in the real world I believe in measurements of reality a lot more than arguments from first-principles.

I think we should insist on a little more rigor and empiricism in this area.

I would love to see claims in academic publication around practicality or reliability justified in the same way we justify performance claims—by doing it. I would be a lot more likely to believe an academic distributed system was practically feasible if it was run continuously under load for

a year successfully and if information was reported on failures and outages. Maybe that isn't feasible for an academic project, but few other allegedly scientific academic disciplines can get away with making claims about reality without evidence.

More broadly I would like to see more systems whose *operation* is verifiable. Many systems have the ability to log out information about their state in a way that makes programmatic checking of various invariants and guarantees possible (such as consistency or availability). An example of such an invariant for a messaging system is that all the messages sent to it are received by all the subscribers. We actually measure the truth of this statement in real-time in production for our Kafka data pipeline for all 450 topics and all their subscribers. The number of corner-cases one uncovers with this kind of check, run through a few hundred use cases and a few billion messages per day is truly astounding. I think this is a special case of a broad class of verification that can be done on the running system that goes far far deeper than what is traditionally considered either monitoring or testing. Call it unit testing in production, or self-proving systems, or next generation monitoring, or whatever, but I think this kind of deep verification is something that makes turning the theoretical claims a design makes into measured properties of the real running system.

Likewise if you have a “NoSQL vendor” I think it is reasonable to ask them to provide hard information on customer outages. They don't need to tell you who the customer is, but they should let you know the observed real-life distribution of MTTF and MTTR they are able to achieve, not just highlight one or two happy cases. Make sure you understand how they measure this, do they have automated test load that runs or just wait for people to complain? This is a reasonable thing for people paying for a service to ask for. To a certain extent if they provide this kind of empirical data it isn't clear why you should even *care* what their architecture is beyond intellectual curiosity.

Distributed systems solve a number of key problems at the heart of scaling large websites. I absolutely think this is going to become the default approach to handling state in internet application development. But no one benefits from the kind of irrational exuberance that currently surrounds the “big data” or nosql systems communities. This area is experiencing a boom—but nothing takes us from boom to bust like unrealistic hype and broken promises. We should be a little more honest about where these systems already shine and where they are still maturing.

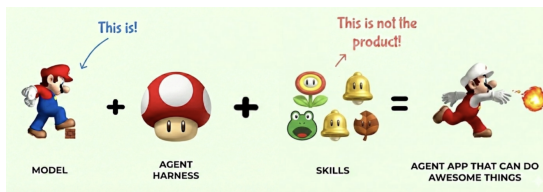
Postscript: This blog seems to have resurfaced 7 years later. I actually still agree with much of it, though the dilemma is much less important. Distributed systems operations is hard, but you can now avoid it by just getting your distributed systems as a service. For example, Confluent

offers Kafka as a service with a contractual uptime SLA. Ops remains hard, but for distributed data systems you can make the people who wrote the software deal with the problem, and focus your time on your special sauce.

It's-a Me, Agentic AI

HAN, NOT SOLO · 01 MAR 2026 · [SOURCE](#)

Agentic AI is a fairly recent development that combines reasoning ([OpenAI, 2024](#)) tool use ([Schick et al, 2023](#)) in the same AI model. But an agentic AI system is not just the model, but also the harness, the environments, the tools, rewards, evaluations, and all of the infrastructure to support it. In this post, let's use the top grossing franchise, Mario, to tell this story for understanding how agentic AI models are developed, how agent harnesses work, and how reinforcement learning ties everything together. If you survived World 8-4 as a kid, you already have the intuition for building agentic AI systems.



alt text

Small Mario is the **base pretrained model**. He's just come out of pretraining on a massive corpus of platform game physics. He can walk. He can jump. He can move left and right. These are his base capabilities, the raw knowledge compressed from the training data.

But Small Mario is fragile. One hit from a Goomba and he's dead. He can't break bricks. He can't take damage. He has potential, but he's not yet useful for anything beyond the most trivial tasks.

This is your base LLM fresh off pretraining. It has absorbed enormous amounts of knowledge, it can do next-token prediction, and it can sort-of follow instructions. But ask it to do anything real, reliably, in production, and it falls apart on the first obstacle. One Goomba and it's game over.

The Super Mushroom: Model Harness

Then Mario finds the **Super Mushroom**. He doubles in size. He can now break bricks. He can take a hit without dying. He goes from fragile to capable.

The Super Mushroom is the **model harness**, the entire infrastructure and post-training pipeline that transforms a base model into something production-ready. This includes:

- **Instruction tuning** (RLHF, DPO, etc.) to make the model actually follow directions
- **System prompts** that define personality and constraints
- **Safety guardrails** so it doesn't run off cliffs
- **Memory and context management** so it remembers where it's been
- **Tool-use training** so it knows power-ups exist and how to grab them

Without the Super Mushroom, Mario is a liability. With it, he's a platform for greatness. Similarly, without the model harness, a base LLM is a research artifact. With it, it's a product.

The Super Mushroom doesn't change WHO Mario is. It changes what he can SURVIVE.
The model harness doesn't change the model's core knowledge. It changes what the model can handle in production.

Now here's where it gets interesting. Super Mario can pick up **power-ups** that give him entirely new capabilities:

Power-Up	Mario Ability	Agent Equivalent
Fire Flower	Throw fireballs to defeat enemies at range	Code execution - reach out and run code to solve problems the model can't solve with text alone
Frog Suit	Swim through water levels with ease	Web search - navigate and retrieve information from an environment the model wasn't trained on
Raccoon Leaf	Fly and reach otherwise inaccessible areas	File system access - explore and modify the codebase, reach files the model couldn't otherwise touch
Hammer Suit	Throw hammers, defeat almost anything	Shell/terminal access - the most powerful tool, can interact with the full system
Star	Temporary invincibility, blow through everything	Extended thinking/reasoning mode - brute force through complex problems at higher compute cost

Cape	Sustained flight and spin	MCP servers - sustained, extensible access to
Feather	attack	external services and APIs

Each power-up doesn't replace Mario's core abilities. Mario still walks and jumps. The power-ups **extend** what he can do. A Fire Flower Mario can still jump on Goombas, but now he can also shoot fireballs at Piranha Plants hiding in pipes.

This is exactly how agent tools work. The LLM still does what LLMs do: reasoning, language understanding, and planning. Tools extend the model's reach into environments it can't operate in alone. An LLM can't execute Python by itself, just like Mario can't throw fireballs without a Fire Flower. But give it the right tool, and suddenly the problem space opens up.

And critically, **Mario has to learn WHEN to use each power-up**. Frog Suit is amazing in water levels, useless on land. Fire Flower is great against Goombas, pointless against Thwomps. The model needs to learn tool selection, knowing which tool to reach for in which context. This is one of the hardest parts of building agentic systems.

One power-up deserves special attention: the **Star**. When Mario grabs a Star, he becomes invincible. He plows through Goombas, Koopa Troopas, Piranha Plants, everything in his path just disintegrates. Nothing can stop him. This is not dissimilar to having an engineering manager who's really good at clearing organizational blockers for their engineers. The Goombas and Piranha Plants of bureaucracy, cross-team dependencies, access requests, and priority conflicts just melt away. Star power is temporary and expensive, but when you need to blast through a critical path, nothing else comes close.

The Mushroom Kingdom: Environments and Tasks

Now let's talk about the world Mario operates in. Every level in the Mushroom Kingdom is an **environment**, and every environment is composed of the same building blocks:

Level Element	Environment Equivalent
Bricks	Structured data, APIs, databases that can be broken open to reveal resources
? Blocks	Unknown information sources, sometimes containing exactly what you need
Pipes	Entry points to sub-tasks, function calls, or deeper exploration
Coins	Incremental progress signals, intermediate rewards
Goombas	Common obstacles, predictable errors, edge cases
Koopa Troopas	Recyclable obstacles, errors whose solutions can be reused as tools

Piranha Plants	Traps hiding in seemingly safe passages, hallucination triggers
Vines	Hidden paths to bonus areas, unexpected shortcuts
Clouds / Platforms	Abstractions and scaffolding that support traversal
Pits	Catastrophic failures, unrecoverable errors

Every level remixes these elements differently. World 1-1 is simple, a few Goombas, some bricks, a clear path to the flag. World 8-4 is a maze of pipes, hidden paths, and a boss fight with Bowser. Same building blocks, radically different difficulty.

Throughout each level, Mario interacts with the environment as **tool use**. He enters pipes to warp from one place to another, the equivalent of an API call that transports you to an entirely different context. He bumps ? Blocks from below to discover power-ups, new agent skills materializing from the environment when you know where to look. He breaks bricks to clear paths or reveal hidden rewards, structured data yielding its value when you apply force in the right direction. He stomps on a Koopa shell and kicks it forward, turning an obstacle into a projectile that clears a line of Goombas, repurposing error outputs as inputs to solve downstream problems. The environment isn't just a backdrop. It's a toolbox.

Having an environment is not enough. We need to define **what we want to achieve** from playing the game. These are the **tasks**.

Mario can play the same level with completely different objectives. Complete the level. Complete the level in the shortest amount of time. Get the highest score. Collect the most coins. Accumulate the most 1-up lives. Find all the hidden rewards. Stomp on every last Goomba. Each objective produces a fundamentally different play style from the same environment.

This is exactly the task definition problem in agentic AI. The same model operating in the same environment can be pointed at wildly different goals. “Summarize this codebase” and “refactor this codebase” use the same files, the same tools, the same context, but they require entirely different strategies. The task is what transforms an environment from a sandbox into a mission.

The Flagpole: Evaluations and Reward Modeling

At the end of every level, there's a **flagpole**. Mario jumps on it, pulls down the flag, and receives a reward. The higher he grabs the flag, the bigger the reward. Some levels end with a boss fight against Bowser, where the reward is freeing a Toad (or eventually, the Princess).

This is the **reward signal** in reinforcement learning. But here’s the critical question: how do we actually **measure** how well the game was played? This is **evaluation**, or **reward modeling**, and it is where the machine learning engineering discipline really shines. (or research engineer, applied AI engineer.)

The evaluation could be the raw score. It could be the number of 1-ups gained. Coins collected. Goombas stomped. Time remaining. Different paths discovered. Most frequently, it is a **combination** of some or all of the above, weighted and balanced against each other. Do we reward Mario more for speed or for thoroughness? For survival or for aggression? For finding secrets or for staying on the critical path?

Evaluation Metric	Mario Measure	Agent Measure
Speed	Time remaining on the clock	Task completion latency
Score	Points accumulated	Overall output quality
Collection	Coins gathered	Information retrieved, resources used efficiently
Completeness	Hidden blocks found, secrets discovered	Edge cases handled, comprehensive coverage
Efficiency	Enemies defeated per life	Correct tool invocations per task
Exploration	Different paths taken	Novel approaches discovered

Designing these rewards is a strict machine learning engineering discipline. A poorly shaped reward function produces an agent that technically completes tasks but in degenerate ways, like a Mario speedrunner who clips through walls. Impressive, but not what we actually wanted. Reward hacking is the Goodhart’s Law of agentic AI: when a measure becomes a target, it ceases to be a good measure.

Here’s where the full picture comes together. How does Mario learn to complete levels?
Reinforcement learning.

Mario starts each level knowing nothing about its specific layout. He has to:

1. **Observe** the current state, what’s on screen, where the enemies are, what power-ups are available
2. **Decide** on an action based on his policy, jump, run, shoot, or wait
3. **Act** and receive feedback from the environment

4. **Update** his policy based on the outcome

This is the MDP (Markov Decision Process) loop. The same loop described in [Agents Are Workflows](#). The same loop that every agentic AI system runs:

The value of Mario's current state equals the expected immediate reward plus the discounted value of the next state. Should Mario jump NOW to get the coin, or wait and avoid the Goomba? The optimal policy balances immediate rewards against future outcomes.

Through repeated play (training episodes), Mario learns:

- Which obstacles can be jumped on vs. avoided
- When to use power-ups vs. save them
- Which pipes lead to shortcuts vs. dead ends
- How to handle boss fights

An agentic AI model goes through the same process. Through reinforcement learning (PPO, DPO, GRPO, or whatever the latest acronym is), the model learns:

- Which tools to invoke for which subtasks
- When to think longer vs. act immediately
- Which approaches work for which problem types
- How to decompose complex tasks into manageable steps

So who's actually making all of this work? Mario doesn't train himself.

To teach agent Mario to be really good at the game, we employ an **Machine Learning Engineer (MLE)**. The MLE is the game designer and coach rolled into one. They construct mock environments, set up the harnesses and tools, define the tasks that Mario needs to achieve, and most importantly, **design the reward function**. The MLE decides what "good" looks like. Do we reward Mario for speed? Thoroughness? Both? How much? This reward design is the single highest leverage decision in the entire pipeline. Get it right and Mario learns to play beautifully. Get it wrong and Mario learns to exploit glitches.

Once the MLE has designed the training setup, the **Machine Learning Systems Engineers (MLSys)** take over. They are the ones who actually run the show at scale. They set up the environment and agent Mario across hundreds to hundreds of thousands of environments, tasks and iterations. They manage the compute, the distributed training runs, the data pipelines. They collect the reasoning traces, the sequences of observations, actions, and outcomes from every single episode Mario plays. And from these traces, they run the reinforcement learning algorithms that allow agent Mario to learn from experience.

This is the unsexy but critical part. An MLE can design the most elegant reward function in the world, but without MLSys engineers standing up the infrastructure to run millions of training episodes and collect the resulting data, Mario never gets past World 1-1.

Worlds: Agent Frameworks and Domains

The Mushroom Kingdom isn't one level. It's organized into **Worlds**, each with a distinct theme and set of challenges:

World	Theme	Agent Domain
World 1	Grassland, basics	Simple text tasks, Q&A, summarization
World 2	Desert	Data analysis, structured environments
World 3	Water	Web and API navigation
World 4	Giant Land	Large-scale refactoring, migrations
World 5	Sky	Architecture and system design
World 6	Ice	Debugging in fragile or legacy environments
World 7	Pipe Maze	Complex multi-step workflows with dependencies
World 8	Bowser's Castle	Full autonomous task completion with adversarial conditions

Agent frameworks are the **World Maps** - they organize how the agent navigates between levels (subtasks) and worlds (domains). Some frameworks are rigid, you follow the path laid out. Others let you pick your route. The best ones (like how Super Mario Bros. 3 introduced the overworld map) let the agent make strategic choices about which level to tackle next.

But remember: the model is the product. The World Map (framework) is useful scaffolding, but Mario does the actual playing. As models get stronger, they need less scaffolding. Today's World 8 is tomorrow's World 1.

Conclusion

The entire agentic AI stack maps to Mario with surprising fidelity:

Mario	Agentic AI
Small Mario	Base pretrained model
Super Mushroom	Model harness and post-training
Power-ups	Agent tools and skills
Star Power	Engineering manager clearing blockers
Levels	Task environments
Pipes, bricks, shells	Tool use within the environment
Objectives (speed, score, coins)	Task definitions
Flagpole / Boss	Reward signal and evaluation
Reward design	Reward modeling (ML engineering discipline)
Learning to play	Reinforcement learning
MLE as game designer	ML engineer designing tasks and rewards
MLSys running millions of episodes	ML infrastructure for training at scale
World Map	Agent framework
Multiplayer	Multi-agent systems

The next time someone asks you to explain agentic AI, ask them: have you played Mario?

If they can understand that Small Mario needs a mushroom to survive, power-ups to be effective, and practice to master each level, they can understand agentic AI development.

Now go save Princess Peach.

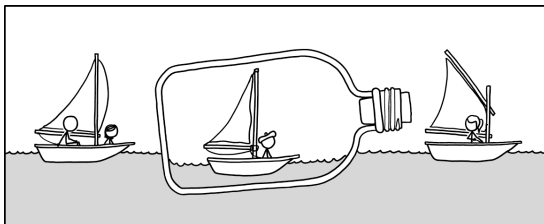
References

```
@article{
  leehanchung,
  author = {Lee, Hanchung},
  title = {It's-a Me, Agentic AI},
  year = {2026},
  month = {02},
  day = {18},
  howpublished = {\url{https://leehanchung.github.io}},
```

```
url = {https://leehanchung.github.io/blogs/2026/02/18/mario-agent  
}
```

Bottle

XKCD.COM · 01 MAY 2026 · [SOURCE](#)



"I know it seems impossible, but the trick is that I sailed in here when I was very young."